Bigram Markov model

A reservoir architecture implementing a soft higher-order Markov model, functioning as a sequence memory. The system has three stages:

1. A delay plus a heteroencoder turn each token pair (x_t, x_{t−1}) into a fresh SHR h_t. The pair is block-coded, so order matters, and there is one random code per bigram.

2. The reservoir is a loop of period 2, with two interleaved reservoirs.

3. The predictor reads x_t and R1 merged into one block.

Requirements

```
<< "Circuits.m"
```

Circuit

```
{
  "$schema": "https://creatingintelligence.org/schemas/circuit-v1.json",

  "hyperparameters": {"default": [2000, 8]},

  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},

    {"component": "delay", "send": [21], "receive": [11]},

    {"component": "heteroencoder", "send": [51], "receive": [11, 21]},

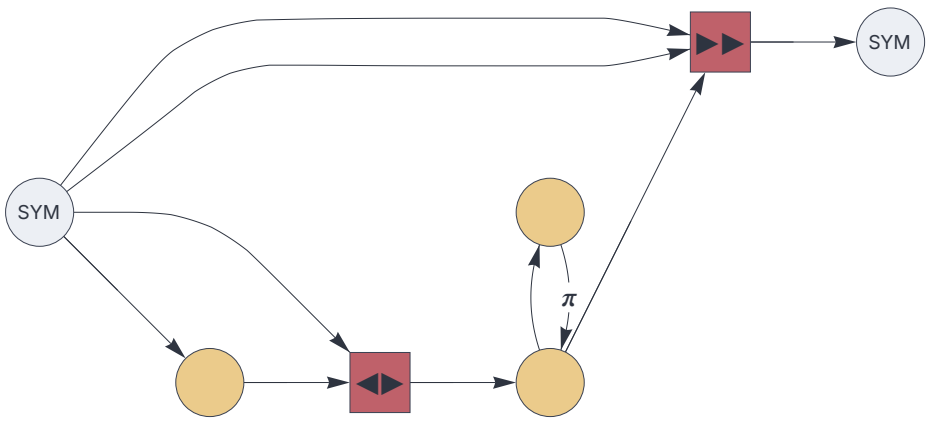
    {"component": "delay", "send": [61],
            "receive": [[51, {"slot": 62, "tags": ["permute"]}]]},

    {"component": "delay", "send": [62], "receive": [61], "rate_limit": 3.0},

    {"component": "predictor", "send": [101], "receive": [11, [11, 61]]},

    {"component": "output", "plugin": "codec", "receive": [101]}
  ]
}
```

```
circ = Circuit[json];
c = circ["function"];
circ["schematics"][ImageSize→500]
```



Sequence learning

```
{Null, Null, p, Null, e, A, p, p, e, A, p,
 p, l, e, A, p, p, l, e, A, p, p, l, e, A, p, p, l, e, A}
```

Dataset

A sequence of 1000 digits:

```
dataset = RandomInteger[ {1, 10}, 1000];
```

Prediction score

```
Do[
    predictions = c /@ dataset;
    Print["    iteration = ", i,
    ", prediction score = ", N[Mean@Boole@MapThread[SameQ,
        {dataset, RotateRight[predictions]}]]],
    {i, 1, 5}];
```

```
iteration = 1, prediction score = 0.097
iteration = 2, prediction score = 0.986
iteration = 3, prediction score = 0.997
iteration = 4, prediction score = 0.999
iteration = 5, prediction score = 1.
```